

Player Performance Prediction (Group 1)

Team Github Repository

<https://github.com/yashs14282001/Group-1-Team-Project.git>

5. Test Plan

1. Purpose and Scope

- Test Plan contains the information regarding the test process to be conducted for the “Cricket Batter and Bowler Performance Prediction Project”. This project predicts the future match performance of any given player depending on his past performance in the chosen format.
- The user decides the format of cricket to be played and the system depends only on the data specific to that format. Database stores some information about the player like name, grade, position, batting form, bowling form, score, run rate, hundreds, fifties, number of wickets, economy rate and player profile.

In Scope

Area	Test Objective
Datasets testing	Validation of player data and datasets of cricket matches
Data format	Verification that system takes Test, ODI or T20 data as per user preference
Batter prediction	Strike rate or batting performance category or validating runs
Bowler prediction	Wickets validity, economy rate or bowling performance category
Models testing	Validating machine learning, neural networks, hybrid and transfer models
App testing	User input validation, prediction validation, error management
Coding	Unit tests, coding standards, code reviews, sign off

Out of Scope

- This test plan does not consider live ball-by-ball prediction, betting tips, guaranteed match result, real-time injury updates, final playing XI confirmation, weather-based prediction, etc.

2. Coding Standard

- All members of the team shall use the same coding style so that all team members' code will be consistent and easily readable. This project will primarily use Python as the programming language; data can be manipulated using either Pandas or NumPy, while Scikit-learn and other machine learning frameworks such as XGBoost or LightGBM can be used to build models.
- Variables and functions will **use lowercase with underscores**, for example:
`player_name, calculate_recent_form()`
- Class names will **use PascalCase**, such as:
`DataPreprocessor`
- File names will **use lowercase with underscores**, such as:

data_cleaning.py
model_evaluation.py

- There will also be a modularized codebase whereby the various file types will represent Data Cleaning, Feature Engineering, Model Building, Model Evaluation, Prediction Making and Application Development. Each of these will have appropriate documentation, including docstrings associated with important functions.
- There will also be error handling for scenarios where there is no dataset, the wrong format is selected by the user, no player data is available, players have not played sufficiently many matches, or predictions fail to execute.

3. Data Quality Testing and Validation

- This project will involve verification of the quality and accuracy of the data provided so that errors in the cricket-related data do not result in inaccurate predictions. There will be at least two different sources of cricket-related datasets used in this project. One dataset will be comprised of player-level data, such as the most recent data provided by the Board of Control for Cricket in India (BCCI).

Data Validation Test

- Data quality will be performed in order to make sure the datasets are suitable for the training and prediction purposes. It is important that the loading process works well, and we load at least two datasets of real matches. The dataset will check required columns like name of players, their role, number of runs, wickets, strike rate and economy rate. Duplicates and missing values will be cleaned to improve the data quality and any incorrect numerical values such as negative runs, wickets, balls and overs will be cleaned as well. Predictive models based on the format of the match – Test, ODI and T20 for classification of data. Missing/unavailable values (e.g. dashes) will be replaced with numeric values.

Format-Based Validation

- Format-based validation is one of the most important tests in this project. The system must not mix performances from different formats.

Scenario	Expected Result
User selects Test	Only Test-related statistics and previous Test matches are used
User selects ODI	Only ODI-related statistics and previous ODI matches are used
User selects T20	Only T20-related statistics and previous T20 matches are used
Invalid format selected	System rejects the input

- For example, if bowler attributes are chosen based on T20 performance, some of the attributes that the model uses would be t20_wickets, t20_economy, recent_t20_wickets and recent_t20_economy. The model does not use performances in ODIs and Test matches for the T20 predictions above.

4. Unit Testing

- Unit testing will be carried out using Pytest. The purpose of unit testing is to test individual functions before combining them into the full system.
- Some of the tests conducted include those related to the functions involved in loading datasets, cleaning data, validating columns, format filtering, recent form calculation, features creation for

batters and bowlers, splitting the data according to time periods, training of models, evaluating them, predicting outcomes based on the models, and input validation for the application.

5. Model Performance Evaluation Against KPIs

- Evaluation of the project will be based on certain key performance indicators and comparisons of three required models: hybrid model, neural network, and transfer/pre-trained model, as well as a baseline model and a traditional ML model like Random Forest, XGBoost, and LightGBM.
- In case of batters, the model will predict runs or classify performance into low, medium, or high. In the case of bowlers, the prediction would be about wickets, economy rate, or classification based on types of bowlers. Regression models will be evaluated based on MAE, RMSE, and R^2 score, while for classification models, the evaluation criteria will be accuracy, precision, recall, and F1 Score.
- KPIs would include using at least two real-world datasets, ability to make format-based predictions, making batter/bowler predictions more accurate than baseline, comparison between the models, boosting model performance using hyperparameter tuning, no data leakage through time-based splitting, development of a working application, and code quality maintained through unit testing and code reviews.

6. User Acceptance Testing

- The User Acceptance Testing process will approve the usability of the final product by the end users which includes the cricket fans, fantasy cricket players, trainers, analysts, students, and project reviewers. UAT scenarios include launching app, selecting Test, ODI or T20 formats, selecting player, selecting batter or bowler prediction, selecting player, opponent and venue, inadequate data, format switching, invalid inputs and prediction explanation display. The expected results are successful launch of the app, successful acceptance of selection of formats, loading of the player profile, show the batting or bowling prediction selected, generate the predictions, generate appropriate warning or error messages, change the results based on format selection, prevent invalid submission and explain the main reasons behind the predictions.
- Example Output: Format: T20 Player: Jasprit Bumrah Type: Bowler Predicted Wickets: 2 Performance Level: High Probability of Prediction: 74% Key Factors: Wickets in last T20 matches, T20 economy rate, performance against opponent team The UAT is successful when the user could do the prediction task easily, the app does not crash during the process, the prediction output is clear and most of the users give positive responses.

7. Code Review and Sign-Off Process

- Code reviews are crucial since they will guarantee that the end result meets all the standards set out in the requirements. This includes correctness, usability, completeness, and being ready for deployment. Developers will carry out the work, perform local tests on their machines, commit their code to a Github branch, get a review from another team member, and only then merge to the master branch. Reviewers will assess coding styles, logic, tests, documentation, exception handling, data validation, model performance, data leakage, app testing, and reproducibility.
- The final sign-off will take place after data validation, format filtering, unit testing, model comparison, KPI testing, app testing success, UAT feedback analysis, and documentation completion.